



Git & GitHub



Cẩm Nang Toàn Diện

Better Code
Better Collaboration
Better Tomorrow!



1. Git là gì?

- Git là một **Distributed Version Control System (DVCS)**.
- Theo dõi các thay đổi trong code của bạn theo thời gian.
- Giúp nhiều lập trình viên làm việc cùng nhau hiệu quả.
- Cho phép bạn quay lại các phiên bản trước đó (history).
- Nhanh, gọn nhẹ và đáng tin cậy.

2. GitHub là gì?



- GitHub là một **cloud platform** dành cho Git repositories.
- Lưu trữ Git repositories trực tuyến.
- Hỗ trợ collaboration, code review và quản lý dự án.

- Cung cấp các công cụ như Issues, Pull Requests, Actions và nhiều hơn nữa.

3. Vì Sao Nên Dùng Git & GitHub?



- ✓ Theo dõi và quản lý các thay đổi trong code của bạn.
- ✓ Collaborate với người khác một cách liền mạch.
- ✓ Làm việc trên nhiều features/branches cùng lúc.
- ✓ Khôi phục sau các lỗi sai (revert về phiên bản cũ).
- ✓ Xây dựng phần mềm tốt hơn, nhanh hơn. ★

4. Khái Niệm Chính



- **Repository (Repo)** : Một thư mục dự án chứa toàn bộ code và history.
- **Commit** : Một bản chụp (snapshot) các thay đổi của bạn.
- **Branch** : Một nhánh phát triển riêng biệt.
- **Merge** : Gộp các thay đổi từ nhánh này sang nhánh khác.
- **Pull / Push** : Lấy thay đổi từ / gửi thay đổi đến một remote repository (VD: GitHub).

5. Git vs GitHub

Git	GitHub
Một công cụ (VCS)	Một nền tảng (Cloud)
Chạy trên máy local của bạn	Lưu trữ Git repositories
Xử lý version control	Hỗ trợ collaboration & chia sẻ
Mã nguồn mở (Open-source)	Dịch vụ dựa trên cloud
Bạn cần cài đặt	Không cần cài đặt (dùng online)

6. Bắt Đầu Thôi



- ✓ Cài đặt Git (<https://git-scm.com>).
- ✓ Tạo một tài khoản GitHub (<https://github.com>).
- ✓ Thiết lập SSH keys (để truy cập an toàn).
- ✓ Khởi tạo một repository và thực hiện commit đầu tiên! ★



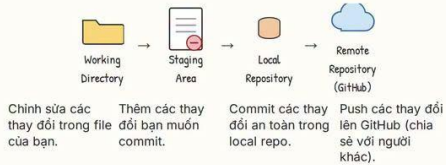
Git & GitHub



Cẩm Nang Toàn Diện

Small steps
every day
lead to big
changes!

1. Quy Trình Git (Tổng Quan)



2. Các Lệnh Git Cơ Bản

`git init` → Khởi tạo một Git repository mới
`git status` → Kiểm tra trạng thái các thay đổi
`git add .` → Thêm tất cả thay đổi vào staging area
`git add <file>` → Thêm file cụ thể vào staging
`git commit -m "msg"` → Commit các thay đổi kèm message
`git log` → Xem lịch sử commit
`git remote -v` → Hiển thị các remote repositories
`git push origin <branch>` → Push lên GitHub
`git pull origin <branch>` → Pull các thay đổi mới nhất
`git clone <url>` → Clone một repository
`git branch` → Liệt kê tất cả các branch
`git checkout <branch>` → Chuyển sang một branch

3. Các Khái Niệm Git Quan Trọng

- Repository**: Một dự án cùng với toàn bộ history của nó.
- Commit**: Một điểm lưu (snapshot) các thay đổi.
- Branch**: Một nhánh phát triển riêng biệt.
- Merge**: Gộp các thay đổi từ nhánh này sang nhánh khác.
- Remote**: Phiên bản repository của bạn được host trên GitHub.
- Clone**: Sao chép một repository từ remote về local.
- Pull**: Fetch và merge các thay đổi từ remote.
- Push**: Tải các thay đổi local lên remote.
- Fork**: Tạo một bản sao repository của người khác.
- Pull Request (PR)**: Đề xuất các thay đổi của bạn để được merge. ★

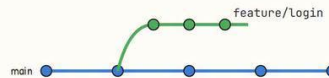
4. Branching & Merging

Vì Sao Cần Branch?

- ✓ Làm việc trên feature mới mà không phá vỡ code chính.
- ✓ Test và thử nghiệm tự do.
- ✓ Giữ cho main branch luôn ổn định.

Lệnh Thường Dùng

`git branch <name>` → Tạo một branch mới
`git checkout <name>` → Chuyển sang branch
`git merge <name>` → Merge vào branch hiện tại
`git branch -d <name>` → Xóa một branch



5. Remote & Collaboration

`git remote add origin <url>` → Thêm remote repository
`git push origin <branch>` → Push code lên GitHub
`git pull origin <branch>` → Lấy các thay đổi mới nhất
`git fetch origin` → Fetch thay đổi (không merge)
`git remote -v` → Xem chi tiết remote

Collaboration Flow:

Fork → Clone → Make Changes → Commit → Push → Create Pull Request

6. .gitignore (Vì Sao Quan Trọng)

- Cho Git biết những file/folder nào cần bỏ qua.
- Giữ cho repository của bạn luôn gọn gàng.
- Ngăn việc push các file không cần thiết hoặc nhạy cảm.

Ví dụ (.gitignore)

```
# Bỏ qua node modules
node_modules/
# Bỏ qua các file log
*.log
# Bỏ qua các file environment
.env
```

- ✓ node_modules/
- ✓ app.log
- ✓ .env
- ✓ .DS_Store
- ✓ build/
- ✓ dist/

7. Các Trường Hợp Thường Gặp



Bắt Đầu Dự Án Mới

```
git init
git add .
git commit -m "Initial commit"
```



Làm Việc Trên Một Feature

```
git checkout -b feature/login
# chỉnh sửa code
git add . && git commit -m "..."
```



Cập Nhật & Đồng Bộ

```
git checkout main
git pull origin main
git merge feature/login
```



Đóng Góp Cho Open Source

Fork repo → Clone về máy
 Tạo branch → Chỉnh sửa
 Push → Tạo Pull Request



Cẩm Nang Toàn Diện

Small consistent actions in Git today, big impact tomorrow!

1. Staging Area Trong Git (Giải Thích)



2. Thư Mục .git (Đằng Sau Hậu Trường)

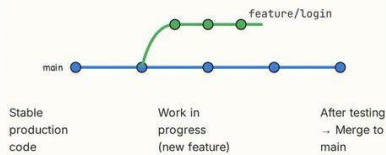
Thư mục `.git` chứa toàn bộ dữ liệu và history quan trọng của repo bạn.

- `objects/` → Tất cả commit, tree, blob
- `refs/` → Các reference (branch, tag)
- `HEAD` → Trỏ đến branch hiện tại
- `config` → Cấu hình repository
- `index` → Thông tin staging area
- `logs/` → Log của mọi hoạt động

💡 Bạn thường không cần đụng vào thư mục này trực tiếp.

3. Mô Hình Branching Của Git

Branch giúp bạn làm việc trên nhiều thứ mà không ảnh hưởng đến code chính.



4. Quy Trình Thường Dùng (Phát Triển Feature)



♥ Luôn cập nhật branch của bạn với main trước khi merge.

5. Hoàn Tác Thay Đổi (Quan Trọng)

Tình Huống	Lệnh	Chức Năng
Thay đổi chưa staged	<code>git restore <file></code>	Bỏ thay đổi trong working directory
Thay đổi đã staged	<code>git restore --staged <file></code>	Unstage file
Commit cuối (giữ thay đổi)	<code>git reset --soft HEAD~1</code>	Undo commit, giữ thay đổi ở staged
Commit cuối (bỏ thay đổi)	<code>git reset --hard HEAD~1</code>	Undo commit, bỏ hết thay đổi
Commit đã push	<code>git revert <commit_id></code>	Tạo commit mới để undo thay đổi

★ Cẩn thận với `--hard` ⚠ nó xóa thay đổi vĩnh viễn!

6. Ví Dụ .gitignore

Vì Sao Dùng `.gitignore`?

- ✓ Giữ repo luôn gọn gàng
- ✓ Tránh push log, build, secrets, dependencies
- ✓ Tiết kiệm dung lượng và giảm conflict

```
# ví dụ .gitignore
node_modules/
.DS_Store
*.log
build/
dist/
.env
venv/
*.pyc
```

7. Best Practices

- ✓ Commit các thay đổi nhỏ và có ý nghĩa.
- ✓ Viết commit message rõ ràng.

Fix login bug
- ✓ Pull trước khi push.
- ✓ Giữ main branch ổn định.
- ✓ Dùng branch cho feature mới/fix lỗi.
- ✓ Review code trước khi merge.

8. Tính Năng GitHub Thường Dùng

Issues

Theo dõi bug, task và feature request.

Pull Requests (PR)

Đề xuất thay đổi và thảo luận trước khi merge.

Actions

Tự động hóa build, test và deploy workflow.

9. Tóm Tắt Nhanh

- 📁 **Thực hiện thay đổi** → Chỉnh sửa trong working directory
- ➕ **Thêm thay đổi** → Chuyển vào staging area
- ✅ **Commit thay đổi** → Lưu vào local repository
- 🌐 **Push thay đổi** → Chia sẻ lên GitHub



Cẩm Nang Toàn Diện

Every commit
is a step
towards growth!
☺

1. Local vs Remote Repository

Local Repository	Remote Repository
Tồn tại trên máy local của bạn.	Được host trên GitHub (cloud).
Bạn có toàn quyền kiểm soát.	Truy cập được bởi mọi người có quyền.
Commit chỉ ở local.	Dùng cho collaboration và backup.
Người khác không nhìn thấy.	

2. Trạng Thái File Trong Git

Untracked: File mới, Git chưa track.

Unmodified: File đã track, không thay đổi.

Modified: Đã track, có thay đổi nhưng chưa staged.

Staged: Thay đổi đã thêm vào staging area.

Committed: Thay đổi đã lưu vào local repository.

Think: Working Directory → Staging Area → Local Repo

3. Các Lệnh Git Quan Trọng (Cheat Sheet)

`git init` → Khởi tạo repository mới
`git clone <url>` → Clone một remote repository
`git status` → Kiểm tra trạng thái thay đổi
`git add <file>` → Thêm file vào staging area
`git add .` → Thêm tất cả thay đổi vào staging
`git commit -m "msg"` → Commit thay đổi kèm message
`git log` → Xem lịch sử commit
`git diff` → Xem thay đổi (chưa staged)
`git diff --staged` → Xem thay đổi (đã staged)
`git branch` → Liệt kê tất cả branch
`git branch <name>` → Tạo một branch mới
`git checkout <name>` → Chuyển sang branch
`git checkout -b <name>` → Tạo & chuyển sang branch mới
`git merge <name>` → Merge branch vào branch hiện tại
`git pull origin <branch>` → Fetch + Merge từ remote
`git push origin <branch>` → Push thay đổi lên remote
`git remote -v` → Hiển thị các remote repositories

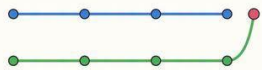
4. Chiến Lược Branching (Quick Guide)

	Main Branch	Code ổn định, sẵn sàng production.
	Feature Branch	Làm việc riêng trên feature mới.
	Hotfix Branch	Fix nhanh lỗi trên production.
	Release Branch	Chuẩn bị cho một bản release mới.
	Develop Branch	Branch tích hợp cho phát triển liên tục.

★ Dùng branch để tách biệt công việc, test an toàn và giữ main branch sạch. ❤

5. Merge vs Rebase

Merge	Rebase
✓ Gộp toàn bộ history.	✓ Di chuyển commit lên trên base khác.
✓ Giữ lại tất cả commit.	✓ History dạng thẳng (linear).
✓ An toàn & dễ dùng.	✓ Commit graph gọn gàng hơn.
✓ Tạo ra một merge commit.	✓ Viết lại history (cẩn thận!).



💡 Quy tắc: Dùng Merge cho branch public, Rebase cho branch cá nhân.

6. Xử Lý Conflict

- ✓ Git sẽ đánh dấu conflict trong file.
- ✓ Mở file → tìm <<<<<<, =====, >>>>>>.
- ✓ Sửa file và giữ lại phần đúng.
- ✓ Lưu file.
- ✓ `git add <file>`
- ✓ `git commit -m "Resolved conflict"`

Conflict Markers
 <<<<<< HEAD
 (Thay đổi của bạn)
 =====
 (Thay đổi đến)
 >>>>>> branch-name
 (Branch khác)

★ Conflict là chuyện bình thường. Xử lý bình tĩnh ☺

7. .gitignore (Người Bạn Tốt Nhất)

Cho Git biết những file/folder nào cần bỏ qua.

Ví dụ:
 node_modules/

8. Pro Tips

Vì Sao Quan Trọng?

- ★ Commit thay đổi nhỏ, có ý nghĩa.
- ★ Viết commit message rõ ràng.
- ★ Pull trước khi push.
- ★ Giữ main branch ổn định.
- ★ Dùng .gitignore.

9. Lời Kết

- ❤ Git giúp bạn theo dõi history.
- ❤ GitHub giúp bạn collaborate.
- ❤ Cùng nhau, chúng biến bạn thành một



Cẩm Nang Toàn Diện

Small steps
every day
lead to big
changes!
😊

1. Thao Tác Với Remote Repository

Lệnh	Chức Năng
<code>git remote -v</code>	Hiển thị các remote repositories.
<code>git remote add origin <url></code>	Thêm một remote repository mới.
<code>git remote remove <name></code>	Xóa một remote.
<code>git remote rename <old> <new></code>	Đổi tên một remote.
<code>git fetch origin</code>	Fetch thay đổi mới nhất từ remote.
<code>git pull origin <branch></code>	Fetch và merge thay đổi.
<code>git push origin <branch></code>	Push thay đổi lên remote.

★ `origin` chỉ là tên mặc định của remote. Bạn có thể đặt tên bất kỳ! 😊

2. Thư Mục .git - Các File Quan Trọng

Thư mục `.git` là trái tim của repository bạn.

- `HEAD` → Trỏ đến branch hiện tại
- `config` → Cấu hình repository & remote
- `objects/` → Lưu mọi object: commit, tree, blob
- `refs/` → Lưu reference đến branch, tag
- `logs/` → Log của mọi hoạt động
- `index` → Staging area (theo dõi thay đổi)

💡 Đừng bao giờ tự ý sửa gì bên trong thư mục `.git`!

3. Stashing Changes

- ✓ Lưu tạm thay đổi mà chưa cần commit.
- ✓ Hữu ích khi cần chuyển branch.
- ✓ Giữ working directory luôn sạch.

`git stash` → Stash các thay đổi
`git stash list` → Liệt kê các stash
`git stash apply` → Áp dụng stash mới nhất
`git stash pop` → Áp dụng & xóa stash
`git stash drop` → Xóa một stash

★ Nghĩ về stash như một "bãi đỗ xe" 🚗 cho công việc của bạn!

4. Tags Trong Git

♥ Tag dùng để đánh dấu điểm cụ thể trong history.

♥ Thường dùng cho các bản release (v1.0, v2.0).

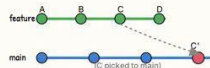
♥ Có thể là lightweight hoặc annotated.

`git tag` → Liệt kê tất cả tag
`git tag <name>` → Tạo một tag
`git tag -a <name> -m "message"` → Tạo annotated tag
`git push origin <tag>` → Push một tag
`git push origin --tags` → Push tất cả tag
 📌 Tag không thay đổi, chúng chỉ trỏ đến một commit! 😊

5. Cherry-pick (Chọn Một Commit)

- Chọn commit cụ thể từ branch này và áp dụng sang branch khác.
- Hữu ích để fix bug.
- Giữ history sạch sẽ.

`git cherry-pick <commit-hash>`
`git cherry-pick <commit1> <commit2> ...`



Great tool, use it wisely! 😊

6. Viết Lại History (Cẩn Thận!)

⚠️ Viết lại history có thể phá hỏng công việc của người khác.

⚠️ Chỉ dùng trên branch local của bạn.

⚠️ Không bao giờ dùng trên branch dùng chung.

`git commit --amend` → Sửa commit gần nhất
`git rebase -i HEAD~n` → Rebase n commit gần nhất
`git reset --soft <commit>` → Giữ thay đổi ở staged
`git reset --mixed <commit>` →

Giữ thay đổi chưa staged

`git reset --hard <commit>` → Bỏ hết mọi thay đổi

★ Nguyên tắc vàng: Đừng viết lại public history! 😊

7. Aliases Hữu Ích (Tiết Kiệm Thời Gian ⌚)

`git config --global alias.co checkout` → `git co`
`git config --global alias.br branch` → `git br`
`git config --global alias.ci commit` → `git ci`
`git config --global alias.st status` → `git st`
`git config --global alias.lg "log --oneline --graph --all"`

→ `git lg`

Alias giúp lệnh Git ngắn gọn và dễ dùng hơn! ♥

8. Mẹo Phòng Vấn

- ✓ Hiểu sâu về Git workflow.
- ✓ Luyện tập branching, merging, rebasing, stashing.
- ✓ Biết sự khác nhau giữa Git và GitHub.
- ✓ Sẵn sàng giải thích tình huống thực tế.

Tình Huống Thực Tế

- Team làm việc trên 1 feature
- Hotfix trên production
- Xử lý merge conflict
- Hoàn tác lỗi sai
- Quản lý release

9. Git vs GitHub

Git	GitHub
Một công cụ (VCS)	Một nền tảng (Cloud)
Chạy trên máy local	Hosting cho Git repositories
Quản lý version control	Hỗ trợ collaboration & chia sẻ

10. Tổng Kết

- ♥ Git giúp bạn kiểm soát mọi thay đổi trong code.
- ♥ GitHub giúp bạn collaborate và chia sẻ với cả thế giới.
- 🔧 Luyện tập mỗi ngày để thành thạo Git & GitHub!



Cẩm Nang Toàn Diện

Keep going!
Small consistent
effort = Big
results
☺

1. Hoàn Tác Trong Git (Các Mức Độ)

Tình Huống	Lệnh	Chức Năng
Thay đổi chưa staged	<code>git restore <file></code>	Bỏ thay đổi trong working directory
Thay đổi đã staged	<code>git restore --staged <file></code>	Unstage file
Commit cuối (giữ thay đổi)	<code>git reset --soft HEAD~1</code>	Undo commit, giữ thay đổi ở staged
Commit cuối (bỏ thay đổi)	<code>git reset --mixed HEAD~1</code>	Undo commit, unstage thay đổi
Commit cuối (xóa thay đổi)	<code>git reset --hard HEAD~1</code>	Undo commit, bỏ hết mọi thay đổi

★ Luôn cẩn thận với `--hard`! Nó xóa thay đổi vĩnh viễn! ☹️

2. .gitignore - Giữ Repo Sạch Sẽ

- Cho Git biết những file/folder nào cần bỏ qua.
- Giữ repo gọn gàng và nhỏ gọn.
- Ngăn việc push các file nhạy cảm hoặc log.
- Giúp tập trung hơn vào code quan trọng.

Ví dụ .gitignore

```
node_modules/
.DS_Store
*.log
dist/
build/
.env
venv/
*.pyc
```

💡 Một file .gitignore tốt giúp bạn tránh được rất nhiều rắc rối! ☺

3. Remote Collaboration Flow



★ Luôn pull thay đổi mới nhất trước khi bắt đầu làm việc.

★ Giao tiếp với team của bạn. ❤️

4. Pull Request (PR) - Sức Mạnh Của Collaboration

- ♥ Đề xuất thay đổi của bạn.
- ♥ Chia sẻ để review và nhận feedback.
- ♥ Thảo luận các cải tiến.
- ♥ Merge sau khi được duyệt.
- ♥ Giữ chất lượng code luôn cao.



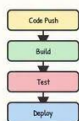
Good PR

- ✓ Tiêu đề rõ ràng
- ✓ Mô tả đầy đủ
- ✓ Liên kết issue
- ✓ Nhỏ gọn & tập trung
- ✓ Dễ review

★ Code chất lượng được xây dựng từ những review chất lượng! 💪

5. Các GitHub Actions Thường Dùng (CI/CD Cơ Bản)

- **Build** → Biên dịch hoặc build dự án.
- **Test** → Chạy automated test.
- **Lint** → Kiểm tra code style và chất lượng.
- **Package** → Chuẩn bị artifact hoặc binary.
- **Deploy** → Deploy lên staging/production.
- **Notify** → Gửi thông báo (Slack, Email).



💡 Tự động hóa việc nhàn chán, tập trung vào xây dựng! 🚀

6. Best Practices (Nguyên Tắc Vàng)

- ✓ Commit các thay đổi nhỏ, có ý nghĩa.
- ✓ Viết commit message rõ ràng, súc tích.
Ví dụ: `Fix login bug in authentication` ❤️
- ✓ Pull trước khi push.
- ✓ Giữ main branch luôn ổn định.
- ✓ Dùng branch cho feature và fix lỗi.
- ✓ Review code trước khi merge.
- ✓ Luôn cập nhật .gitignore.
- ★ Thói quen tốt hôm nay = ít vấn đề hơn ngày mai ☺

7. Sai Lầm Thường Gặp (Tránh Những Điều Này!)

- ❌ Commit các thay đổi lớn hoặc lộn xộn.
- ❌ Push trực tiếp vào main branch.
- ❌ Quên pull trước khi push.
- ❌ Viết commit message không rõ ràng.
- ❌ Bỏ qua merge conflict.
- ❌ Không dùng .gitignore.

Nhớ nhé:
Mắc lỗi cũng không sao, quan trọng là học hỏi và cải thiện! ☺

8. Các Lệnh Một Dòng Hữu Ích

- `git status` → Kiểm tra trạng thái
- `git log --oneline --graph --all` → Xem lịch sử
- `git branch -a` → Liệt kê tất cả branch
- `git stash` → Lưu tạm thay đổi
- `git stash pop` → Áp dụng & xóa stash
- `git remote -v` → Hiển thị các remote
- `git fetch --all` → Fetch tất cả remote
- `git prune` → Xóa các remote branch đã bị xóa

9. Ví Dụ Workflow Thực Tế

Bước	Lệnh
1. Clone repository	<code>git clone <url></code>
2. Tạo branch mới	<code>git checkout -b feature/x</code>
3. Thay đổi & commit	<code>git add . && git commit -m "..."</code>
4. Push & mở PR	<code>git push origin feature/x</code>

10. Lời Kết

- 🚀 Luyện tập tạo nên sự hoàn hảo.
- ♥ Cứ tiếp tục commit, tiếp tục phát triển!

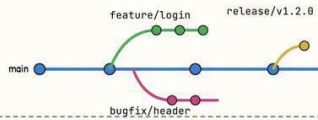


Cẩm Nang Toàn Diện

One step
today,
better developer
tomorrow!

1. Mẹo Branching Nâng Cao

- Luôn giữ main branch ổn định.
- Tạo feature branch từ main mới nhất.
- Xóa branch sau khi đã merge.
- Đặt tên branch có ý nghĩa.



Tên Branch Tốt

- ✓ feature/login-page
- ✓ hotfix/header-bug
- ✓ bugfix/payment-issue
- ✓ release/v1.2.0

★ Một chiến lược branch tốt = History sạch + Team vui vẻ! ☺

2. Thư Mục .git - Đừng Đụng Vào!

Thư mục `.git` lưu trữ toàn bộ dữ liệu quan trọng của repository bạn.

Nó chứa:

- Tất cả commit
- Tất cả branch & tag
- Configuration
- Objects (blobs, trees)
- History và metadata



Đừng xóa hay sửa bất cứ gì bên trong thư mục `.git`. Hãy để Git tự xử lý! ☺

3. Khi Nào Dùng Cái Gì?

Mục Đích	Lệnh Dùng
Bắt đầu dự án	<code>git init</code>
Lưu thay đổi	<code>git add . + git commit -m "msg"</code>
Xem lịch sử	<code>git log --oneline --graph --all</code>
Làm việc trên feature mới	<code>git checkout -b feature/name</code>
Chia sẻ code	<code>git push origin branch-name</code>
Lấy code mới nhất	<code>git pull origin branch-name</code>
Hoàn tác thay đổi local	<code>git restore .</code>
Hoàn tác commit	<code>git reset --soft/mixed/hard</code>
Bỏ qua file	Dùng <code>.gitignore</code>

★ Dùng đúng lệnh vào đúng thời điểm! ☺

4. Merge vs Rebase (So Sánh Nhanh)

Merge (An Toàn & Dễ Dùng)

- ✓ Tạo một merge commit.
- ✓ Giữ toàn bộ history.
- ✓ Không viết lại history.
- ✓ Tốt cho làm việc nhóm.

Rebase (History Sạch)

- ✓ Di chuyển commit lên base khác.
- ✓ History thẳng, gọn gàng.
- ✓ Viết lại history.
- ✓ Chỉ dùng trên branch local.



★ Branch công khai → Merge ✓ Branch cá nhân → Rebase ✓ ☺

5. Tình Huống Thường Gặp

- Có người vừa push code mới ☹️
→ `git pull origin main`
- Đang làm feature mới
→ `git checkout -b feature/xyz`
- Commit cuối bị sai
→ `git commit --amend`
- Lỡ tay xóa mất thay đổi
→ `git reflog` → tìm commit → `git reset --hard <id>`
- Conflict khi merge
→ Resolve → `git add .` → `git commit` ❤️

6. GitHub Tuyệt Vời ✨

- 📁 **Repositories** Lưu trữ dự án của bạn.
- ! **Issues** Theo dõi bug và task.
- 🔗 **Pull Requests** Collaborate & review code.
- 🔧 **Actions** Tự động hóa workflow.
- 📁 **Projects** Quản lý task trực quan.
- 📖 **Wiki** Tài liệu hóa dự án.

GitHub + You = Siêu Năng Lực

- ✓ Backup code của bạn
- ✓ Truy cập từ mọi nơi
- ✓ Collaborate với cả thế giới
- ✓ Showcase công việc của bạn

7. Bí Quyết Từ Chuyên Gia ✨

- Commit nhỏ, commit thường xuyên.
- Viết commit message có ý nghĩa.
- Pull trước khi push.

Kỷ luật nhỏ, kết quả lớn ☺

8. Điểm Nhanh Cho Phòng Vấn 🎯

- Git là gì? → DVCS
- GitHub là gì? → Cloud platform
- Nắm vững các khái niệm cốt lõi



Cẩm Nang Toàn Diện

Little by little
daily practice
leads to huge
improvement!

1. Lệnh Git Nâng Cao (Phải Biết)

`git log --oneline --graph --all` → Xem tất cả branch dạng graph
`git diff` → Xem thay đổi (chưa staged)
`git diff --staged` → Xem thay đổi (đã staged)
`git show <commit-hash>` → Xem chi tiết một commit
`git blame <file>` → Xem ai đã thay đổi từng dòng
`git bisect start` → Tìm commit gây ra lỗi
`git stash branch <name>` → Tạo branch từ stash
`git clean -n` → Dry run: hiện các file chưa track
`git clean -f` → Xóa các file chưa track
`git rebase -i HEAD~n` → Interactive rebase (mạnh mẽ!)

★ Học một ít mỗi ngày, hôm nay biết lệnh, mai thành thạo! ☺

2. .gitignore - Ví Dụ Thông Minh

Nên bỏ qua những gì?

- Dependencies
- Logs
- OS / IDE files
- Environment files
- Build folders

```
# Dependencies
node_modules/
# Logs
*.log
# OS files
.DS_Store
Thumbs.db
# IDE files
.vscode/
.idea/
# Environment
.env
# Build output
dist/
build/
```

Vì sao quan trọng?

- ✓ Giữ repo sạch
- ✓ Tránh push file nhay cảm
- ✓ Giảm dung lượng repo
- ✓ Ngăn conflict không cần thiết ♥

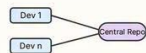


Một repo sạch là một repo hạnh phúc! ☺

3. Các Mô Hình Git Workflow

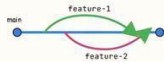
a) Centralized Workflow

Tất cả developer push lên một repository trung tâm.
 Phù hợp cho: Team nhỏ



b) Feature Branch Workflow

Tạo feature branch từ main và merge bằng Pull Request.
 Phù hợp cho: Hầu hết team



c) Git Flow Workflow

Lý tưởng cho dự án lớn với release và hotfix.
 Phù hợp cho: Team lớn / Doanh nghiệp



Chọn workflow phù hợp với team và dự án của bạn. ☺

4. Tính Năng GitHub Nên Dùng

! **Issues** - Theo dõi bug, task và feature request.

📁 **Projects** - Sắp xếp task theo board (kiểu Kanban).

🔗 **Pull Requests** - Collaborate, review và merge code.

🔧 **Actions (CI/CD)** - Tự động hóa build, test và deployment.

📖 **Wiki** - Tài liệu hóa dự án.

💬 **Discussions** - Đặt câu hỏi và chia sẻ ý tưởng.

📢 **Releases** - Công bố phiên bản mới của dự án.

★ Khám phá tính năng, tăng năng suất! ☺

5. Best Practices Cho Commit Message

Commit Message Tốt

- ✓ Dùng imperative mood (Add, Fix, Update).
- ✓ Ngắn gọn, rõ ràng.
- ✓ Giải thích what và why (không phải how).
- ✓ Tham chiếu issue nếu có.

Ví dụ: `feat: add user login api`
 Fixes #123 ♥



Một commit message tốt kể câu chuyện về code của bạn. ☺

Commit Message Chưa Tốt

- ✗ Update stuff
- ✗ fixed bug
- ✗ Changes
- ✗ final working code

6. Tài Nguyên Hữu Ích

📖 **Official Git Book** → git-scm.com/book

📖 **GitHub Docs** → docs.github.com

📖 **Git Cheat Sheet** → education.github.com

🗨 **Oh Shit, Git!** → ohshitgit.com

📖 **GitHub Skills** → skills.github.com

♥ Tài nguyên tốt = Học nhanh hơn! ☺

7. Thói Quen Git Hằng Ngày (Làm Ngay!)

- Pull thay đổi mới nhất trước khi bắt đầu.
- Làm việc trong một branch riêng.

8. Lời Kết

♥ Git giúp bạn theo dõi mọi thay đổi.

🔧 Tiếp Tục Luyện Tập!



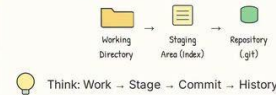
Cẩm Nang Toàn Diện

Progress
every day,
success
every time!
☺

1. Git Internals - Điều Gì Xây Ra?

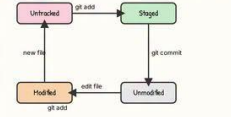
Khi bạn chạy một lệnh Git, nó thao tác chủ yếu trên 3 khu vực:

1. **Working Directory** - Các file hiện tại của bạn.
2. **Staging Area (Index)** - Các file bạn add để chuẩn bị commit.
3. **Repository (.git)** - Toàn bộ commit và history.



2. Chi Tiết Trạng Thái File

Untracked - File mới với Git.
Unmodified - Không có thay đổi từ commit cuối.
Modified - Có thay đổi nhưng chưa staged.
Staged - Đã đánh dấu cho commit tiếp theo.



★ Hiểu rõ các trạng thái, làm chủ Git! ☺

3. Tìm Hiểu Sâu Về Branching

- ✓ Branch cho phép làm việc trên nhiều thứ mà không ảnh hưởng main.
- ✓ Tạo, chuyển, merge, xóa branch dễ dàng.
- ✓ Dùng tên có ý nghĩa.

Mẹo Đặt Tên

- ✓ feature/... (feature mới)
- ✓ hotfix/... (fix khẩn cấp)
- ✓ bugfix/... (fix bug)
- ✓ release/... (bản release mới)



★ Chiến lược branch tốt = Code sạch + Team vui vẻ! ♥

4. Các Chiến Lược Merge

- | Merge | Rebase |
|-------------------------|---------------------------|
| ✓ Merge mặc định. | ✓ Áp lại commit lên trên. |
| ✓ Tạo một merge commit. | ✓ History dạng thẳng. |
| ✓ Giữ toàn bộ history. | ✓ Project graph gọn hơn. |
| ✓ An toàn và dễ dùng. | ✓ Dùng trên branch local. |



5. Hoàn Tác Thay Đổi - Sửa Sai Không Sợ!

- `git checkout -- <file>` → Bỏ thay đổi trong file
- `git reset --soft HEAD~1` → Undo commit, giữ thay đổi ở staged
- `git reset --mixed HEAD~1` → Undo commit, unstage thay đổi
- `git reset --hard HEAD~1` → Undo commit, bỏ hết mọi thay đổi
- `git revert <commit-hash>` → Tạo commit mới để undo
- `git clean -n` → Xem các file chưa track
- `git clean -f` → Xóa các file chưa track

⚠️ Cẩn thận với `--hard` và `git clean -f` ! ☺

6. Best Practices Khi Collaborate Trên GitHub

- ✓ Tạo PR có ý nghĩa.
- ✓ Viết mô tả rõ ràng.
- ✓ Liên kết issue.
- ✓ Yêu cầu review.
- ✓ Xử lý feedback.
- ✓ Squash commit nếu cần.
- ✓ Xóa branch đã merge.

Good Pull Request

- ✓ Tiêu đề rõ ràng
- ✓ Mô tả
- ✓ Mục đích
- ✓ Liên kết issue
- ✓ Screenshot (nếu cần)
- ✓ Nhỏ gọn & tập trung

Collaboration makes us better together! ✨

7. Git Hooks - Tự Động Hóa Tác Vụ

- ✓ Hook là script tự động chạy khi có sự kiện Git nhất định.
- ✓ Dùng cho code quality, test, lint, thông báo và nhiều hơn nữa.

Hook Thường Dùng:

- pre-commit
- commit-msg
- pre-push
- post-merge
- và nhiều hơn...

Ví Dụ Sử Dụng

- ✓ Chạy test trước khi commit
- ✓ Kiểm tra code style
- ✓ Ngăn file quá lớn
- ✓ Gửi thông báo

💡 Tự động hóa và tiết kiệm thời gian!

8. Thư Mục .git - Sức Mạnh Ẩn

Thư mục `.git` lưu trữ toàn bộ dữ liệu của repository.

Nó chứa:

- Objects (commit, tree, blob)
- Refs (branch, tag)
- Logs (history)
- Config (settings)
- Hooks (scripts)
- Và còn nhiều hơn...

9. Git Config - Cài Đặt Của Bạn

Cấu hình Git để cá nhân hóa trải nghiệm của bạn.

Cài Đặt Thường Dùng:

- `user.name`
- `user.email`
- `core.editor`
- `core.autocrlf`
- `push.default`
- và nhiều hơn...

⚙️ Cấu hình càng chi tiết, Git càng "vừa vặn" với bạn!



Cẩm Nang Toàn Diện

Consistency
today,
Mastery
tomorrow!

1. Reset, Revert, Restore - Phân Biệt

Lệnh	Công Dụng	Hiệu Ứng	An Toàn
<code>git reset --soft</code>	Undo commit, giữ thay đổi ở staged	Di chuyển HEAD	★★★★
<code>git reset --mixed</code>	Undo commit, unstage thay đổi	Di chuyển HEAD + unstage	★★★☆☆
<code>git reset --hard</code>	Undo commit, bỏ hết thay đổi	Di chuyển HEAD + xóa thay đổi	★★☆☆
<code>git revert <commit></code>	Undo commit bằng cách tạo commit mới	History vẫn an toàn	★★★★
<code>git restore <file></code>	Bỏ thay đổi trong working directory	Ảnh hưởng working directory	★★★★

💡 Nếu phân vân, dùng revert. Nếu chắc chắn, dùng reset.
Nếu chỉ 1 file, dùng restore! ☺

2. .gitignore - Là Gì & Vì Sao?

- Cho Git biết những file/folder nào cần bỏ qua.
- Giữ repository luôn sạch sẽ.
- Ngăn secrets, log và file hệ thống bị push.
- Cải thiện performance và sự tập trung.

Các Mục .gitignore Thường Gặp

```
# Dependencies
node_modules/
# Logs
*.log
# OS files
.DS_Store
Thumbs.db
# IDE files
.vscode/
.idea/
# Environment
.env
# Build output
dist/
build/
```

★ Một .gitignore nhỏ hôm nay giúp bạn đỡ khổ về sau! ☺

3. GitHub Workflow - Bức Tranh Toàn Cảnh



Điểm Quan Trọng

- ✓ Luôn làm việc trong một branch.
- ✓ Thảo luận, review, phê duyệt.
- ✓ Mở PR sớm để nhận feedback.
- ✓ Merge và ăn mừng! 🎉

4. Hướng Dẫn Viết Commit Message

Commit Message Tốt

- ♥ Dùng imperative mood (Add, Fix, Update).
- ♥ Ngắn gọn, rõ ràng.
- ♥ Giải thích what và why.
- ♥ Tham chiếu issue nếu có.

Commit Message Chưa Tốt

- ✗ Updated stuff
- ✗ Fixed bug
- ✗ Changes
- ✗ Test
- ✗ Final code

Ví dụ: Fix user login validation (BRX-18479)

Mẹo

Một commit message tốt giống như một ghi chú tốt cho chính bạn sau này! ☺

5. Best Practices Cho Pull Request (PR)

- Giữ PR nhỏ gọn và tập trung.
- Tiêu đề và mô tả rõ ràng.
- Liên kết issue liên quan.
- Thêm screenshot nếu thay đổi UI.
- Squash commit trước khi merge (nếu cần).
- Nhẹ nhàng và tôn trọng khi review.

Good PR Checklist

- ✓ Tiêu đề có ý nghĩa
- ✓ Mô tả rõ ràng
- ✓ Liên kết issue
- ✓ Screenshot (nếu cần)
- ✓ Nhỏ gọn & tập trung
- ✓ Sẵn sàng để review

★ PR nhỏ = Reviewer vui vẻ + Merge nhanh! ☺

6. Quy Ước Đặt Tên Branch



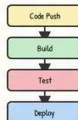
- `feature/<name>` → `feature/login-page`
- `bugfix/<id>-<name>` → `bugfix/10479-login-issue`
- `hotfix/<name>` → `hotfix/critical-bug`
- `release/<version>` → `release/v1.2.0`

Mẹo

- ✓ Nhất quán.
- ✓ Dùng chữ thường và dấu gạch ngang.
- ✓ Ngắn gọn nhưng đủ ý nghĩa.

7. Các GitHub Actions Thường Dùng (CI/Cd)

- **Build** → Biên dịch hoặc build dự án.
- **Test** → Chạy automated test.
- **Lint** → Kiểm tra code style.
- **Package** → Tạo artifact.
- **Deploy** → Deploy đến environment.
- **Notify** → Gửi thông báo.



💡 Tự động hóa việc nhàm chán. Tập trung xây dựng điều tuyệt vời! ☺

8. Bảo Mật & An Toàn

- Không bao giờ commit secrets (API key, password).
- Dùng file `.env` và ignore nó.
- Dùng branch protection rules.
- Yêu cầu review PR.
- Luôn cập nhật dependencies.
- Quét lỗ hổng bảo mật.

Ghi Nhớ

🔒 Bảo mật không phải việc làm một lần. Đó là một thói quen!

★ Code an toàn = Tin tưởng + Yên tâm! ☺